



TRIBHUVAN UNIVERSITY
ASIAN SCHOOL OF MANAGEMENT & TECHNOLOGY

Final Year Project Report
On
“Retail Management System”

Submitted To :
Office of Dean
Institute of Science and Technology

Under the supervision of
Mr. Chakra Rawal

*In partial fulfillment of the requirement for the bachelor's degree in Computer
Science and
Information Technology*

Submitted By :
Sushank Lamsal (79010980)
Manish Karki (79010963)
Sudarshan Magar (79010978)

Chaitra, 2025

SUPERVISOR’S RECOMMENDATION

It is my pleasure to recommend that a report on “Retail Management System” has been prepared under my supervision by Sushank Lamsal, Manish Karki and Sudarshan Magar in partial fulfillment of the requirement of the degree of Bachelor of Science in Computer Science and Information Technology (BSc.CSIT). Their report is satisfactory to process for the future evaluation.

.....

Mr. Chakra Rawal

Supervisor

Asian School of Management & technology

Gongabu, Kathmandu

CERTIFICATE OF APPROVAL

This is to certify that this report has been read and recommended to the Department of Computer Science and Information Technology for acceptance of report entitled “Retail Management System” submitted by Sushank Lamsal, Manish Karki and Sudarshan Pata Magar in partial fulfillment for the degree of Bachelor of Science in Computer Science and Information Technology (BSc.CSIT), Institute of Science and Technology, Tribhuvan University.

.....

Mr. Chakra Rawal

Supervisor

.....

Er. Anil Lal Amatya

Principal

.....

Mr/Mrs.

External Examiner

.....

Mr. Chakra Rawal

Program Co-ordinator

ACKNOWLEDGEMENT

We would like to express our honest gratitude to all those who contributed to this project. Our sincere thanks go to our supervisor and coordinator **Mr. Chakra Rawal**, who provided us with precious guidance and support throughout the project. Their invaluable expertise, encouragement, and support have been a constant source of inspiration. We would also like to thank the Asian School of management and technology College for providing us with the resources and support needed to complete this project. Finally, we would like to extend our gratitude to our families and friends for their support and encouragement throughout the completion of the project.

With respect,

SUSHANK LAMSAL

MANISH KARKI

SUDARSHAN MAGAR

ABSTRACT

The Retail Management System is a web-based platform designed to support digital retail operations with a focus on intelligence, usability, and system efficiency. The application provides essential features for secure user access, structured product presentation, order handling, online payment processing, email-based communication, and personalized user account management, ensuring a smooth and reliable user experience. A major intelligent feature of the system is the integration of AI-based visual search and image recognition technology. This functionality enables users to search for products by uploading images instead of relying on text-based queries. Using deep learning based image recognition techniques, the system analyzes visual characteristics such as shape, color, and patterns to identify and match similar products from the database. This approach simplifies product discovery, enhances user convenience, and improves engagement, particularly when users are unfamiliar with product names or descriptions.

Keywords: *MVT, ORM, Django, Python, PyTorch, Agile, JavaScript, visual search*

TABLE OF CONTENT

TITLE PAGE	ii
SUPERVISOR'S RECOMMENDATION	ii
CERTIFICATE OF APPROVAL	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
TABLE OF CONTENT	vi
LIST OF ABBREVIATIONS	vii
LIST OF FIGURES	viii
LIST OF TABLES	ix
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	2
1.2 Problem Statement	2
1.3 Objectives.....	2
1.4 Scope and Limitation	3
1.5 Development Methodology	4
1.6 Report Organization	5
CHAPTER 2: BACKGROUND STUDY & LITERATURE REVIEW	6
2.1 Background Study	6
2.2 Literature Review	7
CHAPTER 3: SYSTEM ANALYSIS	9
3.1 System Analysis	9
3.1.1 Requirement Analysis	9
3.1.2 Feasibility Study	14
3.1.3 Analysis.....	16
CHAPTER 4: SYSTEM DESIGN	19
4.1 Design Overview	19
4.2 Algorithm Details	24
CHAPTER 5: IMPLEMENTATION AND TESTING	31
5.1 Implementation	31
5.2 Testing.....	34
5.3 Result Analysis	41
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATION	44
6.1 Conclusion	44
6.2 Future Recommendations	45
REFERENCES	47
APPENDICES	48

LIST OF ABBREVIATIONS

API	Application Programming Interface
BSC	Bachelor of Science
CSIT	Computer Science and Information Technology
DB	Database
DRY	Don't Repeat Yourself
GPU	Graphics Processing Unit
HLD	High Level Design
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
ICT	Information and Communication Technology
JWT	JSON Web Token
KISS	Keep It Simple, Stupid
ML	Machine Learning
MVT	Model View Template
NLP	Natural Language Processing
OAuth	Open Authorization
ORM	Object Relational Mapper
OTP	One Time Password
PWA	Progressive Web Application
Redis	Remote Dictionary Server
SQL	Structured Query Language
URL	Uniform Resource Locator
VPS	Virtual Private Server

LIST OF FIGURES

Figure 1.1: Agile Development Model	5
Figure 3.1: Use Case Diagram of Retail Management System	10
Figure 3.2: Gantt Chart	15
Figure 3.3: Class Diagram of Retail Management System	16
Figure 3.4: Sequence Diagram of Retail Management System.....	17
Figure 3.5: Activity Diagram of Retail Management System.....	18
Figure 4.1: High-Level System Design of Retail Management System	19
Figure 4.2: Refinement of Class Diagram	20
Figure 4.3: Refinement of Sequence Diagram	21
Figure 4.4: Refinement of Activity Diagram	22
Figure 4.5: Component Diagram of Retail Management System	23

LIST OF TABLES

Table 3.1: Use Case Scenario for User Operations	11
Table 3.2: Use Case Scenario for Admin Operations	12
Table 4.1: Dataset Attribute Description	30
Table 5.1: Test for Account Module	34
Table 5.2: Test for Store & Product Module.....	35
Table 5.3: Test for Cart Module	36
Table 5.4: Test for Order & Payment Modul	37
Table 5.5: Test for Visual Search Module.....	38
Table 5.6: System Test Cases	39
Table 5.7: Performance Evaluation Results of the Visual Search System.....	43

CHAPTER 1

INTRODUCTION

1.1 Introduction

The rapid growth of digital technology has significantly transformed the retail industry, shifting traditional business operations toward online and web-based platforms. E-commerce systems have become essential tools for managing products, customers, and transactions efficiently while providing convenience and accessibility to users. As consumers increasingly prefer online shopping, there is a strong need for reliable retail management systems that can support day to day retail operations in a secure, organized, and user-friendly manner.

A Retail Management System is designed to help retailers manage their online business activities such as product listing, customer registration, shopping cart management, order processing, and payment handling through a centralized web application. By automating these processes, the system reduces manual effort, minimizes errors, and improves overall operational efficiency. At the same time, customers benefit from easy product browsing, smooth checkout processes, and personalized account management.

The proposed project focuses on developing a web-based e-commerce Retail Management System using modern web technologies. The system aims to provide essential functionalities including user authentication, product and category management, cart and order handling, payment integration, and email-based notifications. The application is structured using a modular approach to ensure maintainability, scalability, and clear separation of responsibilities across different components of the system.

In addition to standard e-commerce functionalities, the system also includes an image-based product search feature. This feature allows users to search for products by uploading an image instead of typing product names. The system analyzes basic visual characteristics such as color, shape, and patterns to find and display similar products available in the store. This functionality improves product discovery, especially for users who may not know the exact name or description of a product.

This project demonstrates the practical application of web development concepts, database management, and software engineering principles along with the integration of ML features in web applications.

1.2 Problem Statement

The rapid expansion of online shopping has transformed customer expectations, with users increasingly demanding fast, intelligent, and visually driven product search experiences; however, many retail systems used by small and medium-sized businesses still rely solely on text-based search, creating challenges when customers wish to search using images such as photos seen on social media, in magazines, or captured personally without knowing the exact product name or description. Additionally, many existing retail platforms operate through disconnected systems where inventory, orders, payments, customer accounts, and notifications are managed separately or manually, resulting in data inconsistencies, slow processing, duplicate data entry, and poor coordination between front-end user interfaces and back-end operations. Furthermore, most current systems lack advanced features such as real-time error detection, user behavior analytics, secure payment integration, and personalized shopping experiences, which limits retailers' ability to resolve issues promptly, understand customer preferences, reduce cart abandonment, and build customer trust; therefore, there is a strong need for a unified, intelligent, and scalable web-based retail management system that addresses these challenges in an integrated manner.

1.3 Objectives

- To create a centralized web platform that allows customers to browse products, manage shopping carts, place orders, make secure payments, receive email notifications, and handle personal accounts.
- To implement visual search functionality so users can upload product images to find matching or similar items, making product discovery faster and more intuitive.

1.4 Scope and Limitation

The scope of Retail Management System are as follows:

- End-to-End Retail Operations: Covers the complete lifecycle from product cataloging with variations to order fulfillment, invoicing, and status tracking.
- AI-Driven Visual Discovery: Integrates deep learning (MobileNetV3) and vector search (FAISS) to enable image-based product recommendations beyond traditional text search.
- Secure Financial Transactions: Implements HMAC-signed payment gateway integration (eSewa) and Cash on Delivery with automated transaction verification.
- Dynamic Inventory Management: Supports real-time stock updates, category hierarchy management, and multi-attribute product variations (size, color).
- Customer Engagement Loop: Facilitates verified purchase reviews, rating systems, and personalized user dashboards for order history.
- Scalable Architecture Foundation: Built on Django's modular app structure with Sentry integration for error monitoring and potential horizontal scaling.

Every system has error so this system has also limitations are as follows:

- Static Content Delivery: Relies on server-side rendered HTML templates without a decoupled frontend framework (e.g., React/Vue), limiting dynamic UI interactivity.
- Single-Model Visual Search: The visual search engine is restricted to MobileNetV3 features, lacking support for multi-model ensembles or custom dataset fine-tuning without re-indexing.
- Limited Payment Gateway Diversity: Currently supports only eSewa and COD, lacking immediate integration with international gateways like Stripe or PayPal.
- Session-Based Cart Persistence: Cart data is tied to user sessions or authentication states, potentially leading to data loss if session cookies expire before login.
- Absence of Real-Time Analytics: Lacks built-in real-time dashboarding for sales trends, inventory forecasting, or customer behavior heatmaps.
- Basic Notification System: Email notifications are synchronous and limited to order confirmations, lacking SMS integration or push notification support.

1.5 Development Methodology

The Retail Management System is developed using the **Agile Development Methodology**, emphasizing flexibility, collaboration, and iterative progress. The development process is organized into sprints, where each sprint focuses on delivering key functionalities while ensuring a user-centric design.

Iteration 1: The first phase establishes the core infrastructure, including Django environment setup, SQLite database schema design, and wireframe creation. A minimum viable product (MVP) is developed covering custom user registration, email-based authentication, and basic product catalog management with category hierarchies.

Iteration 2: This iteration focuses on the core shopping module, implementing dynamic product search and filtering, product variation management (size/color), and a session-based shopping cart that handles real-time subtotal and tax calculations for authenticated users.

Iteration 3: The third iteration integrates the complete order processing workflow, including checkout logic, atomic transaction handling, and secure payment gateway integration (eSewa with HMAC verification and Cash on Delivery) to ensure financial data integrity.

Iteration 4: In this phase, user engagement features are implemented, including a verified purchase review and rating system, personalized user dashboards for order tracking, and automated email notifications for order confirmations and status updates.

Iteration 5: The final iteration introduces advanced AI components, specifically the visual search engine using PyTorch, MobileNetV3 for feature extraction, and FAISS for efficient vector similarity matching. The system undergoes comprehensive testing for performance, security vulnerabilities, and scalability before deployment.

By following this iterative Agile methodology, the Retail Management System ensures continuous improvement, rapid delivery of features, and adaptability to user feedback throughout the development process.

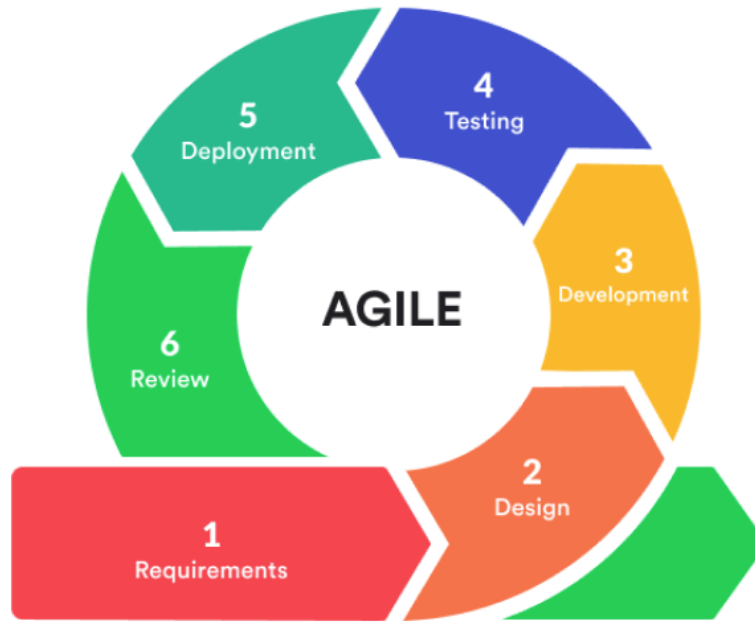


Figure 1.1 Agile Development model

1.6 Report Organization

The Report Organization describes how the study is structured and conducted. This report is divided into six chapters, each addressing different aspects of the study. Chapter One presents the introduction, statement of the problem, objectives of the study, scope and limitations, development methodology and organization of the study. Chapter Two focuses on the background study and review of related literature, providing the necessary theoretical foundation. Chapter Three discusses system analysis, including requirement analysis, feasibility study, data modeling, and process modeling of the system. Chapter Four deals with the design of the system, particularly the system architecture and its components. Chapter Five explains the implementation phase, the algorithms used, and the tools applied, along with the testing of the system both as individual units and as a 4 complete system. Finally, Chapter Six provides the conclusions drawn from the study, highlights its limitations and offers recommendations for future improvements.

CHAPTER 2

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

The retail industry has undergone a significant transformation from traditional brick-and-mortar stores to sophisticated e-commerce platforms, driven by the need for 24/7 accessibility and global reach. Modern consumers expect seamless digital experiences that replicate the convenience of physical shopping while offering broader product selections. However, traditional text-based search mechanisms often fail to capture user intent when customers lack specific vocabulary or are inspired by visual aesthetics rather than keywords. This gap highlights the necessity for intelligent systems that can bridge the divide between visual inspiration and product discovery, making the integration of advanced search capabilities a critical component of contemporary retail management.

Recent advancements in Artificial Intelligence and Deep Learning have revolutionized how users interact with online catalogs, particularly through Computer Vision technologies. Convolutional Neural Networks (CNNs), such as MobileNetV3, enable efficient extraction of high-dimensional feature vectors from images, allowing systems to understand visual content similarly to human perception. When combined with optimized similarity search libraries like FAISS (Facebook AI Similarity Search), retailers can implement real-time visual search engines that identify similar products based on image uploads rather than text queries. This technological shift moves beyond simple keyword matching, offering a more intuitive shopping experience where an image of a desired item can instantly retrieve relevant inventory from a vast database.

Despite these technological strides, many existing retail solutions remain fragmented, offering either robust inventory management or basic storefronts without integrated AI capabilities. Furthermore, emerging markets often lack localized payment integrations, such as eSewa in Nepal, which are essential for regional commercial viability. There is a distinct need for a unified Retail Management System that combines secure, role-based

authentication, comprehensive order processing, and localized payment gateways with cutting-edge visual search technology. This project addresses these challenges by developing a holistic platform that not only streamlines backend operations for store owners but also enhances the frontend customer journey through intelligent, image-driven product discovery.

2.2 Literature Review

Recent research underscores the transformative impact of Artificial Intelligence (AI) on e-commerce and retail management, marking a paradigm shift from reactive inventory systems to proactive, user-centric platforms[1]. Traditional text-based search mechanisms, while foundational, often create friction in the customer journey when users lack precise terminology or seek inspiration rather than specific items. Studies indicate that keyword dependency limits product discovery, leading to higher bounce rates and missed sales opportunities[2]. In response, AI-powered visual search has emerged as a critical innovation, leveraging deep learning and computer vision to decode visual attributes such as color palettes, textures, shapes, and stylistic patterns. By allowing users to upload images as queries, these systems bridge the semantic gap between human visual perception and database retrieval, significantly enhancing the intuitiveness of the shopping experience[3].

The technical efficacy of visual search is well-documented in contemporary literature, with Convolutional Neural Networks (CNNs) serving as the backbone for feature extraction. Research demonstrates that lightweight architectures like MobileNetV3 and robust models like ResNet can generate high-dimensional embedding vectors that accurately represent image content[4]. When paired with efficient similarity search libraries such as FAISS (Facebook AI Similarity Search), these systems achieve millisecond-level retrieval times even across massive catalog[5]. Industry analyses from major retailers like Amazon (StyleSnap), ASOS, and Pinterest validate these findings, reporting that AI-driven visual search can increase user engagement, reduce cart abandonment, and boost conversion rates by up to 30%. Furthermore, academic discourse extends beyond search functionality to encompass broader AI integrations, including personalized recommendation engines, dynamic pricing strategies, and predictive inventory optimization, all of which contribute to a more resilient and responsive retail ecosystem[6].

Despite these advancements, a significant gap exists in the literature regarding the holistic integration of advanced AI capabilities with comprehensive retail operational workflows, particularly for small to medium-sized enterprises (SMEs). Existing studies often treat visual search as a standalone module or focus exclusively on backend inventory management, neglecting the synergistic potential of a unified platform. There is limited documentation on architectures that seamlessly combine secure, region-specific payment gateways (such as eSewa), robust order lifecycle management, and real-time stock synchronization with deep learning-based visual discovery. Moreover, few frameworks address the specific challenges of implementing these technologies within an Agile development lifecycle that prioritizes iterative feedback and scalability[7]. The Retail Management System addresses these deficiencies by developing a cohesive, web-based solution that integrates state-of-the-art visual search technology with end-to-end retail operations, offering a practical, scalable model that enhances both operational efficiency and customer satisfaction in emerging markets.

CHAPTER 3

SYSTEM ANALYSIS

3.1 System Analysis

System analysis involves studying the system in detail to understand how it works and what improvements are needed. It focuses on identifying problems, gathering data, and separating the system into smaller parts to make it easier to study. By analyzing each part and how they connect, we can clearly see what the system is expected to do. In this way, system analysis acts as a method of problem solving, helping to check whether each component works properly and contributes toward the overall goal of the system.

3.1.1 Requirement Analysis

To develop the Retail Management System, we identified both functional and non-functional requirements that the system must fulfill.

3.1.1.1 Functional Requirement

Functional requirements specify the essential services the system will provide.

- **User Account Management**

The system allows users to create, access, and manage their accounts securely.

- **Product Browsing and Viewing**

The system enables users to browse, search, and view products through an organized and user-friendly interface.

- **Online Shopping and Order Processing**

The system allows users to add products to the cart, place orders, and manage order processing efficiently.

- **Payment Handling**

The system securely processes online payments and records payment transactions.

- **Image-Based Product Search**

The system enables users to upload product images to search for visually similar products using image recognition techniques.

- **E-mail Based Notification**

The system sends email notifications to users regarding important activities such as account verification, order confirmation, and order status updates.

- **Administrative Management**

The system provides an administrative interface for managing users, products, orders, categories, and other system operations.

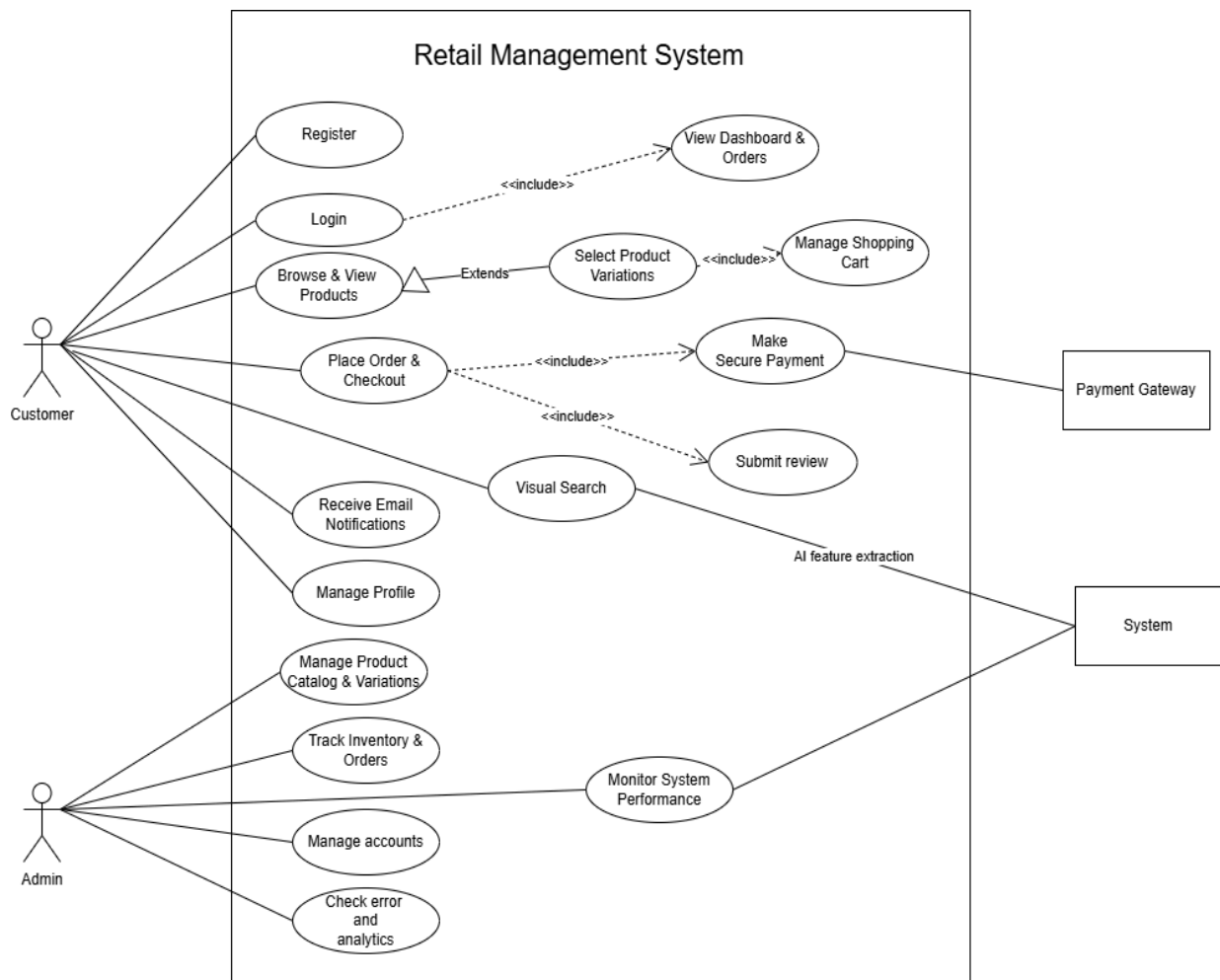


Figure 3.1 : Use Case Diagram of Retail Management System

Field	Description
Use Case	End-to-End User Operations
Primary Actor	Customer
Secondary Actor	Online Store System (Django), AI Visual Search Engine (MobileNetV3/FAISS), Email Service, Payment Gateway (eSewa)
Flow of Events	1. Customer registers or logs in using secure email authentication. 2. Customer searches for products via text keywords, category filters, or by uploading an image for AI visual search. 3. System displays product details and a list of visually similar items generated by the AI engine. 4. Customer selects specific product variations (size/color) and adds them to the shopping cart. 5. System validates real-time stock availability and recalculates totals. 6. Customer confirms shipping details and completes payment via eSewa or Cash on Delivery. 7. System automatically creates the order record, decrements inventory, and triggers confirmation emails to the customer and admin. 8. Customer views order history, tracks status, and submits a review for delivered items
Pre-Condition	1. Customer has a valid account and is logged in. 2. The product catalog is active and the visual search index is built. 3. The payment gateway (eSewa) is operational.
Post-Condition	1. Order is created with "New" or "Confirmed" status. 2. Inventory levels are accurately updated. 3. Payment transactions are recorded and verified. 4. Confirmation notifications are sent to both the customer and the store administrator.
Quality Requirements	1. Security: All transactions must use HTTPS; payment signatures must be verified via HMAC-SHA256. 2. Data Integrity: Order creation and stock reduction must occur in an

Field	Description
	atomic database transaction to prevent overselling. 3. Performance: Visual search results must render within 2 seconds; page loads under 1 second. 4. Reliability: System must handle concurrent user requests without race conditions. 5. Responsiveness: Email notifications must be dispatched within 60 seconds of order placement.

Table 3.1: Use Case Scenario for User Operations

Field	Description
Use Case	Admin Operations
Primary Actor	Super Admin (Platform Manager)
Secondary Actor	Django Admin Panel, Email Service, Sentry (Error Tracking
Flow of Events	1. Admin logs into the secure Django admin panel. 2. Admin manages product catalog (add/edit/delete), categories, and variations. 3. Admin processes orders by updating status (New → Accepted → Completed) or handling cancellations. 4. Admin moderates user reviews and resolves customer complaints. 5. Admin triggers the <code>build_index</code> command to update the AI visual search database. 6. Admin monitors system logs via Sentry and generates sales/inventory reports.
Pre-Condition	1. Admin has valid superuser credentials. 2. Admin is authenticated and authorized to access the dashboard.
Post-Condition	1. Product inventory and order statuses are up-to-date. 2. Visual search index is synchronized with the latest products. 3. Inappropriate content is removed; reports are generated.

Field	Description
Quality Requirements	1. Security: Strict role-based access control (RBAC); all actions logged. 2. Integrity: Stock updates must be atomic to prevent data conflicts. 3. Responsiveness: Admin dashboard actions must reflect immediately. 4. Reliability: System must prevent deletion of products with active orders.

Table 3.2: Use Case Scenario for Admin Operations

3.1.1.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes, constraints, and performance expectations of the system. These requirements define how well the system should operate rather than what it does.

- **Usability**

The system is designed to be user-friendly and intuitive, allowing users with basic computer and internet knowledge to interact with the system easily without requiring extensive training.

- **Performance**

The system provides acceptable response times during user interactions such as browsing, searching, and transaction processing to ensure a smooth user experience.

- **Security**

The system is designed to protect user data, transaction details, and system resources through secure authentication, authorization, and data handling mechanisms.

- **Scalability**

The system is capable of handling future growth in users, products, and transactions without significant changes to its core architecture.

- **Reliability**

The system operates consistently with minimal downtime and should maintain data integrity even under unexpected conditions.

- **Maintainability**

The system is designed in a modular manner to simplify maintenance, debugging, and future enhancements.

- **Compatibility**

The system functions correctly across common web browsers and devices to ensure broad accessibility for users.

3.1.2 Feasibility Analysis

The feasibility study evaluates the practicality and viability of the proposed Retail Management System from multiple perspectives to ensure that the project can be successfully developed and implemented within the given constraints.

i) Technical Feasibility

The system is technically feasible as it is developed using well established and widely supported technologies such as HTML, CSS, JavaScript, jQuery, Bootstrap for the frontend, and Django with PostgreSQL for the backend and database management. These technologies are stable, open-source, and suitable for developing scalable and secure web-based applications. The development team has sufficient knowledge and access to the required tools, frameworks, and development environments, making the technical implementation achievable.

ii) Economic Feasibility

The system is economically feasible because it relies primarily on open-source technologies, which significantly reduces development and licensing costs. No expensive hardware or proprietary software is required for development or deployment. The system is developed using standard computing resources, making it cost-effective and suitable for an academic project with limited financial constraints.

iii) Legal Feasibility

The system is legally feasible as it does not violate any known legal or regulatory requirements. The technologies and frameworks used are open-source and legally permissible for academic and educational use. The system is designed to handle user data responsibly, following general data protection and privacy principles, ensuring compliance with applicable legal standards.

iv) Operational Feasibility

The system is operationally feasible as it is designed to be user-friendly and easy to operate for both customers and administrators. The web-based nature of the system allows users to access it through standard web browsers without the need for specialized training. This ensures smooth adoption and effective day-to-day operation once the system is deployed.

v) Schedule Feasibility

The system is schedule feasible as it is developed within the allocated academic timeline by following an Agile development approach. The use of modular development and iterative planning allows tasks to be managed efficiently, reducing the risk of delays and ensuring timely completion of the project milestones. For our project, we have used Gantt Chart.

Given below is a Gantt Chart describing the schedule for this project:

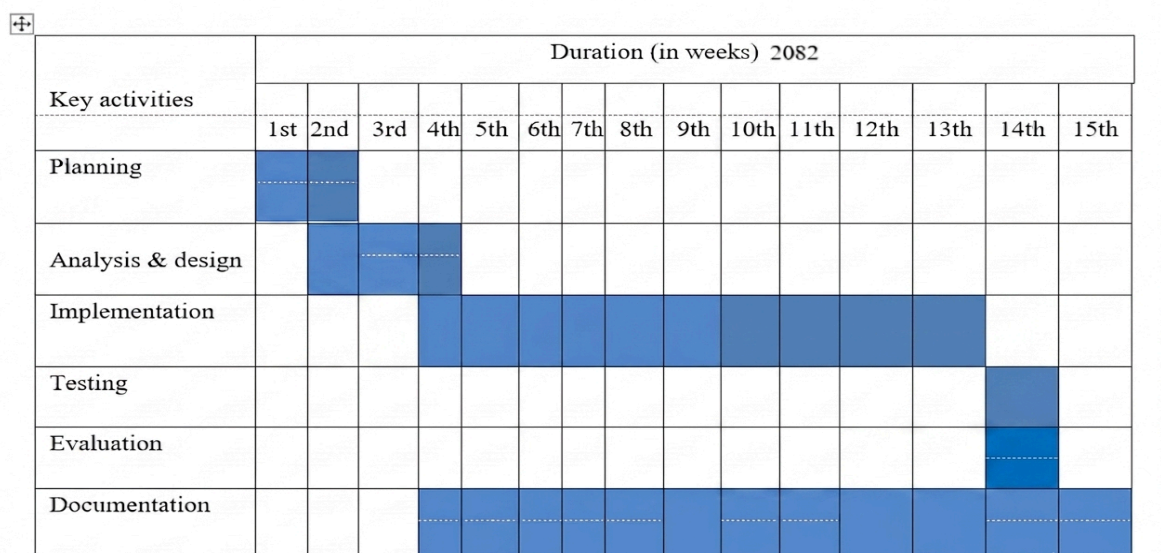


Figure 3.2 : Gantt Chart

3.1.3 Analysis

3.1.3.1 Object Modeling using Class Diagram

The Class Diagram serves as the static blueprint of the Retail Management System, defining the core structure and relationships between key entities. At the center is the Account class, which manages user authentication and profiles, linking to CartItem for session-based shopping and Order for transaction history. The Product class acts as the primary inventory entity, associated with Category for hierarchical organization and Variation to handle specific attributes like size and color. A critical relationship exists between Product and the VisualSearchEngine component, where product images are processed into feature vectors stored in a FAISS Index. The Order class aggregates OrderProduct entries and links to a Payment class, which encapsulates transaction details and verification statuses for gateways like eSewa.

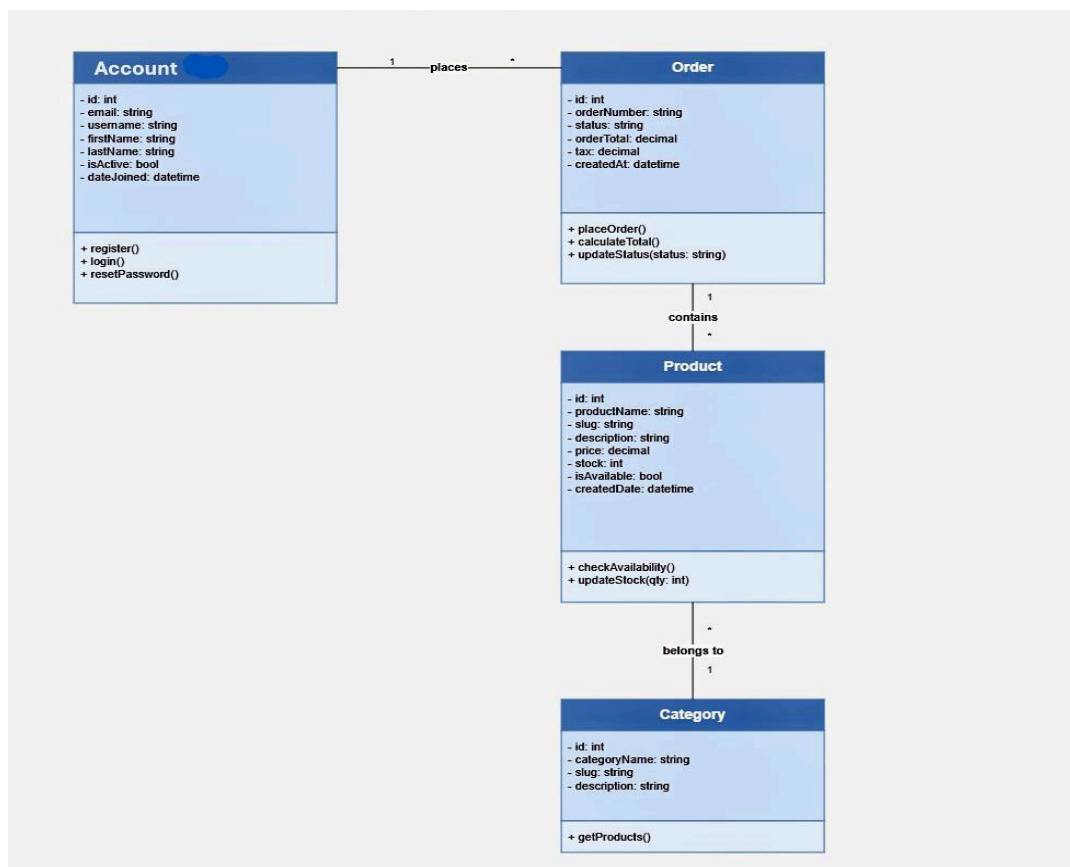


Figure 3.3: Class Diagram of Retail Management System

3.1.3.2 Dynamic Modeling using Sequence Diagram

The Sequence Diagram illustrates the dynamic interaction between system objects over time, specifically focusing on the critical "Checkout and Payment" workflow. It depicts the chronological flow of messages starting from the Customer initiating an order, passing through the CartController to validate stock, and invoking the OrderManager to create a transaction record. The diagram highlights the synchronous call to the Payment Gateway for secure signature generation and redirection, followed by the asynchronous callback verification. Finally, it shows the Inventory Service updating stock levels and the NotificationService dispatching confirmation emails, ensuring that all components collaborate atomically to maintain data consistency during the purchase process.

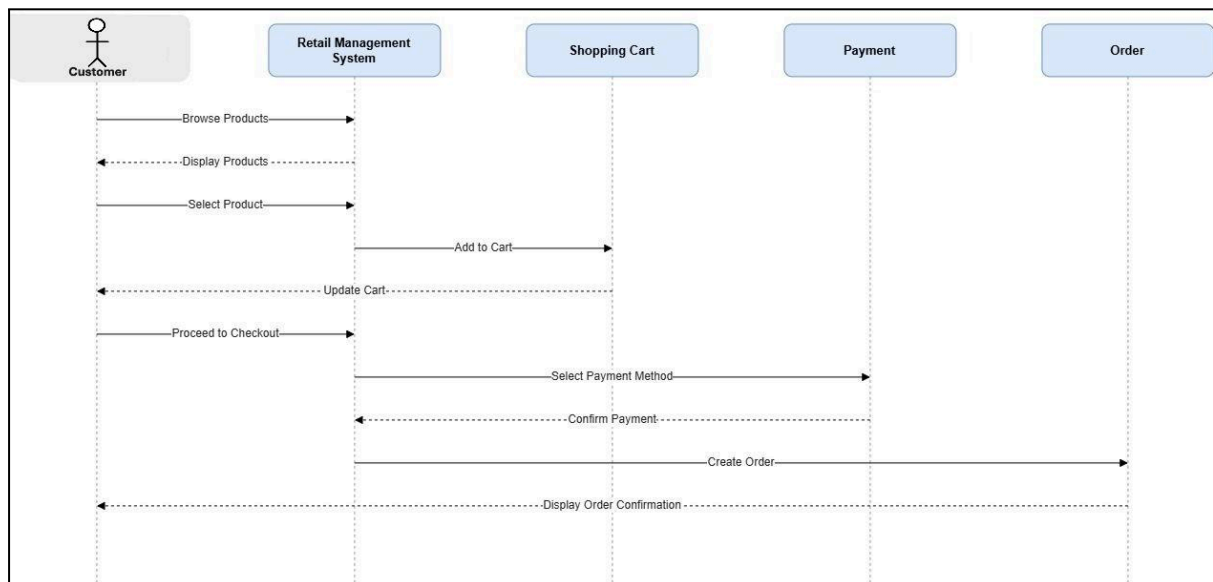


Figure 3.4: Sequence Diagram of Retail Management System

3.1.3.3 Process Modeling using Activity Diagram

An Activity Diagram illustrates the workflow of activities performed within a system. It represents the sequence of actions, decision points, parallel activities, and the overall flow of control from the beginning of a process to its completion. For the Retail Management System, the activity diagram models the product purchasing process. It illustrates how a customer browses products, adds items to the shopping cart, proceeds to checkout, selects a payment

method, completes payment, and receives order confirmation. The diagram also includes decision points such as stock availability and payment verification to demonstrate how different execution paths are handled during the ordering process.

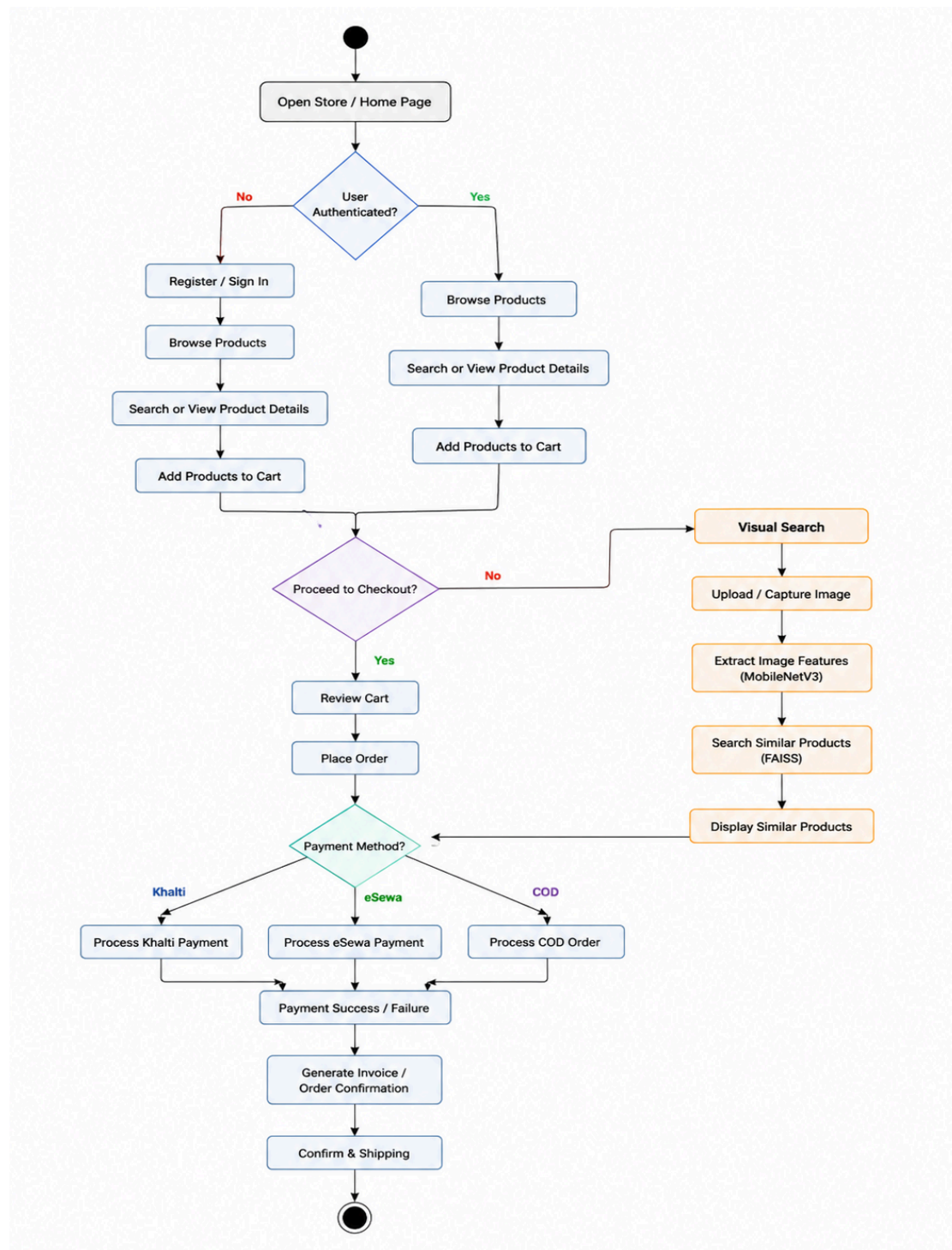


Figure 3.5: Activity Diagram of Retail Management System

CHAPTER FOUR

SYSTEM DESIGN

4.1 Design

System Design includes the means and methodologies to improve the management and control of the software development process. It includes structuring and simplifying the process using several diagrams and standardizing the development process by specifying the required activities and techniques to be implemented.

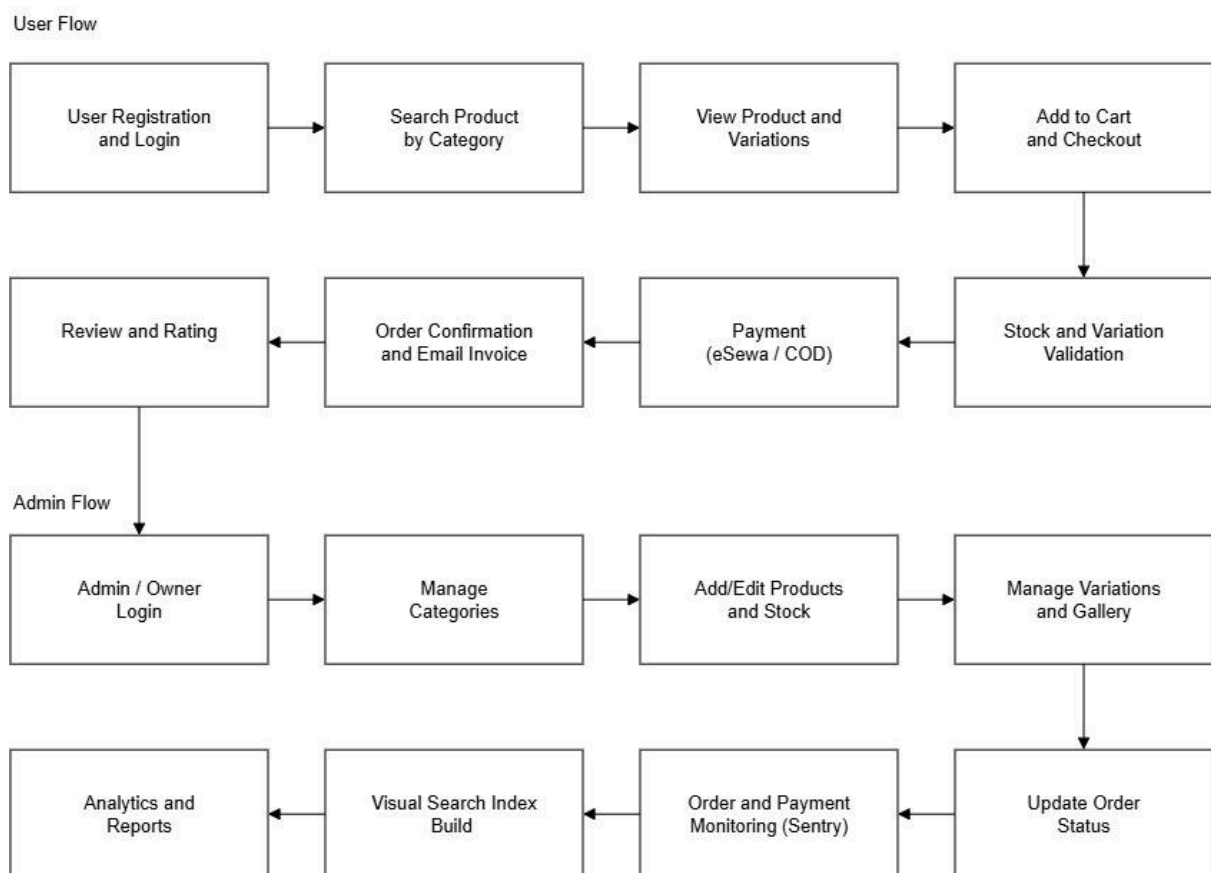


Fig 4.1: High-Level System Design of Retail Management System

4.1.1 Refinement of Class Diagram

The refinement of the class diagram is shown in the figure below:

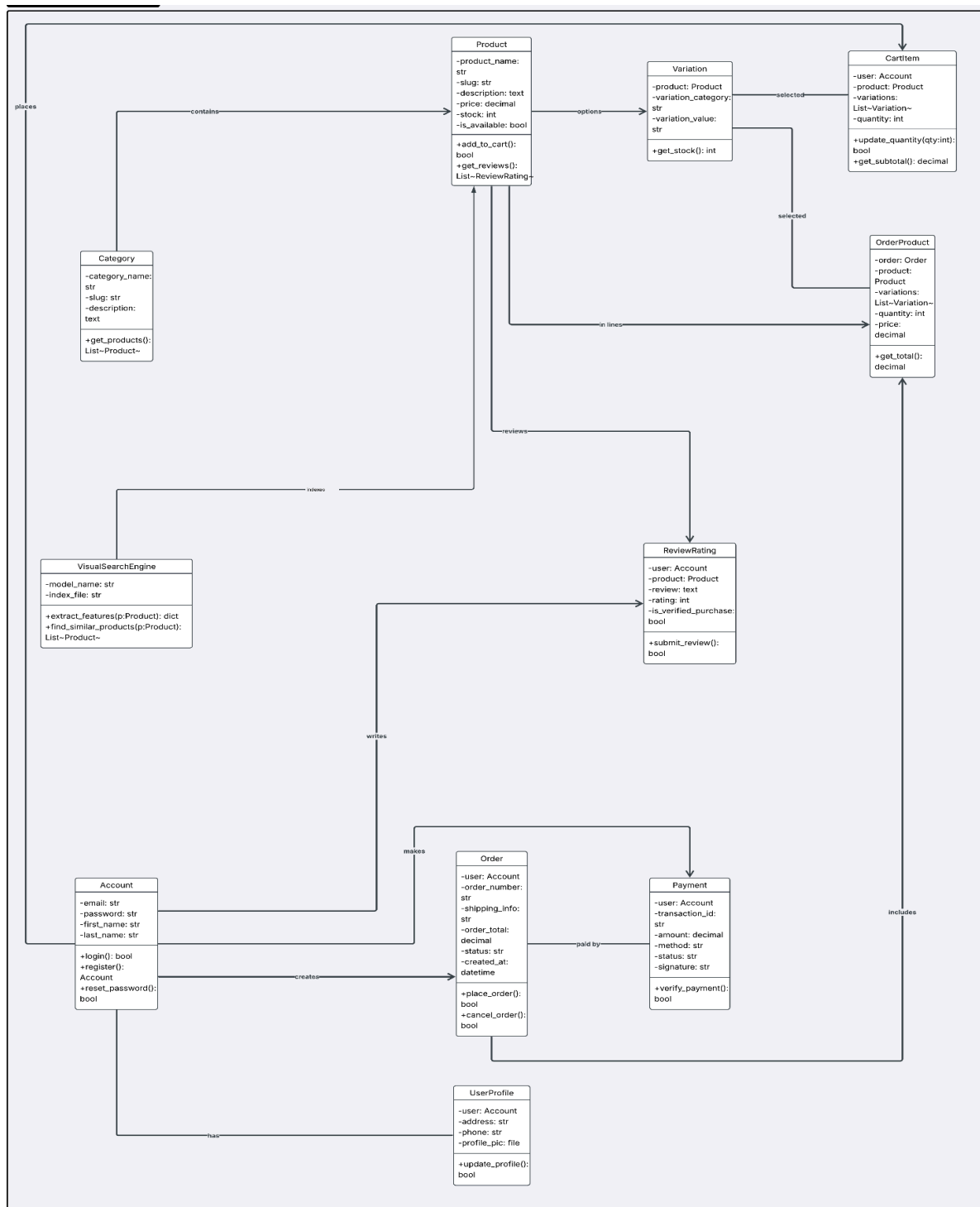


Figure 4.2: Refinement of Class Diagram

The refined class diagram extends the analysis class diagram by incorporating implementation-specific details of the Retail Management System. In addition to the primary

classes identified during the analysis phase, the refined diagram includes important attributes, operations, data types, multiplicities, and relationships that reflect the actual Django application. It provides a more detailed representation of the system's architecture and serves as a blueprint for software implementation.

The refinement also illustrates the interactions among classes such as Account, UserProfile, Category, Product, Variation, CartItem, Order, OrderProduct, Payment, and ReviewRating. Relationships such as one-to-one, one-to-many, and many-to-many associations are clearly specified along with class responsibilities. This detailed model improves the understanding of data flow, supports maintainability, and ensures that the software design accurately represents the implemented retail management system.

4.1.2 Refinement of Sequence Diagram

The refinement of the sequence diagram is shown in the figure below:

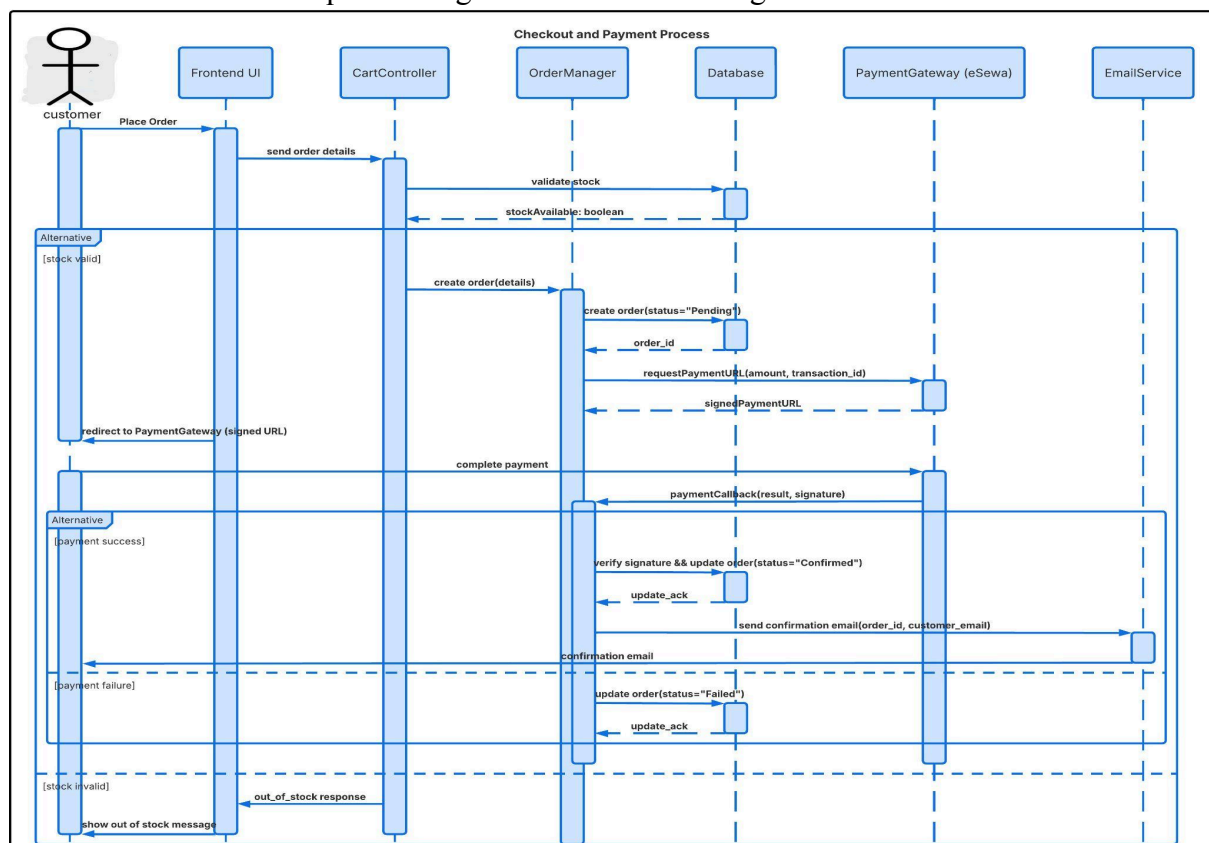


Figure 4.3: Refinement of Sequence Diagram

The refined sequence diagram provides a detailed view of the interactions between the system components during the execution of a specific use case. It expands the analysis sequence diagram by incorporating implementation-level elements such as Django views, models, payment services, and database operations. The diagram clearly illustrates the chronological exchange of messages between the user and various system components involved in processing a request.

For the Retail Management System, the refined sequence diagram represents the order placement and payment workflow. It demonstrates how user requests are handled, validated, processed through the checkout module, verified by the payment gateway, and stored in the database before generating an order confirmation. This detailed interaction model serves as a guide for implementing and verifying the system's behavior.

4.1.3 Refinement of Activity Diagram

The refinement of the sequence diagram is shown in the figure below:

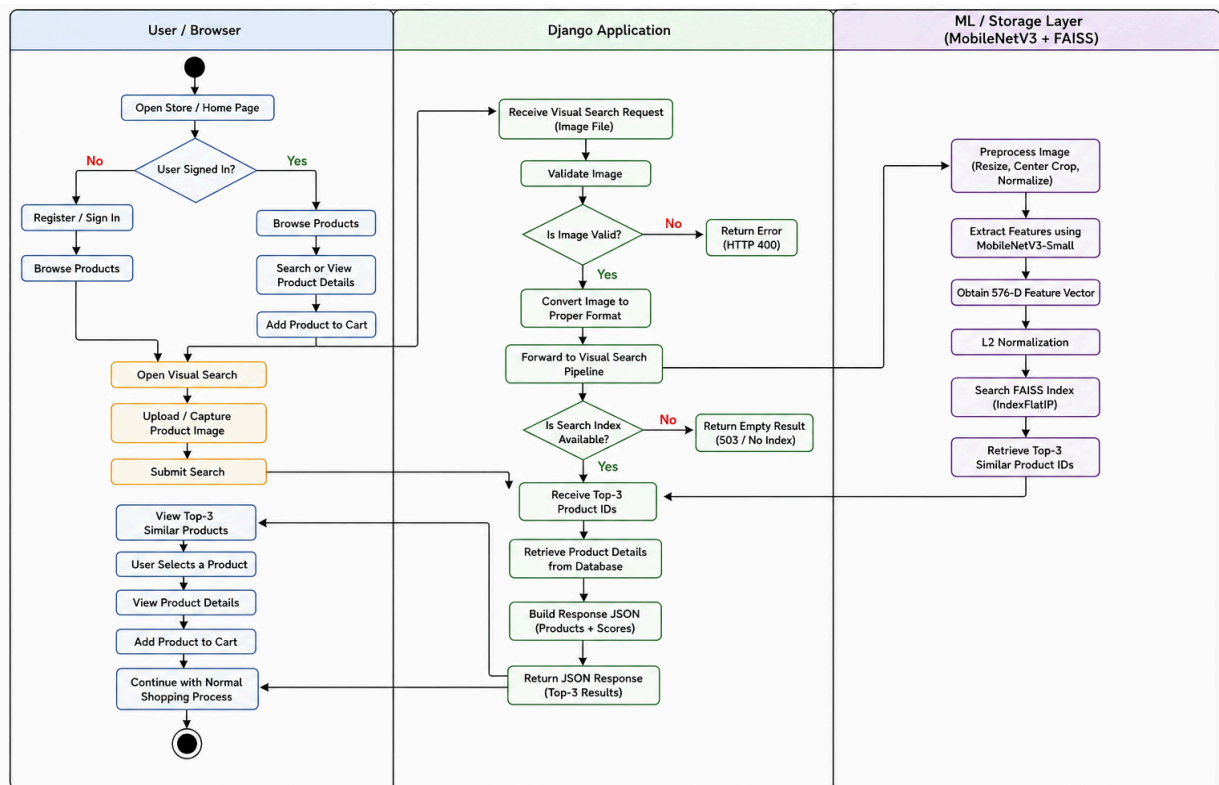


Figure 4.4: Refinement of Activity Diagram

The refined activity diagram enhances the analysis activity diagram by providing a more detailed representation of the workflow and decision-making process within the system. It includes implementation-specific activities such as input validation, stock availability checking, payment verification, order creation, inventory updates, and invoice generation. These additional activities reflect the actual processing logic of the Retail Management System.

The refined diagram also illustrates the different execution paths that may occur during order processing, including successful transactions, payment failures, and insufficient stock conditions. By modeling these alternative flows and decision points, the diagram provides a comprehensive understanding of the system's operational behavior and supports the development, testing, and maintenance of the application.

4.1.4 Component Diagram

A component diagram illustrates how a system is divided into distinct parts (components) and how these parts interact with one another. It emphasizes the modular design of the system, showing the interfaces and dependencies among the key elements.

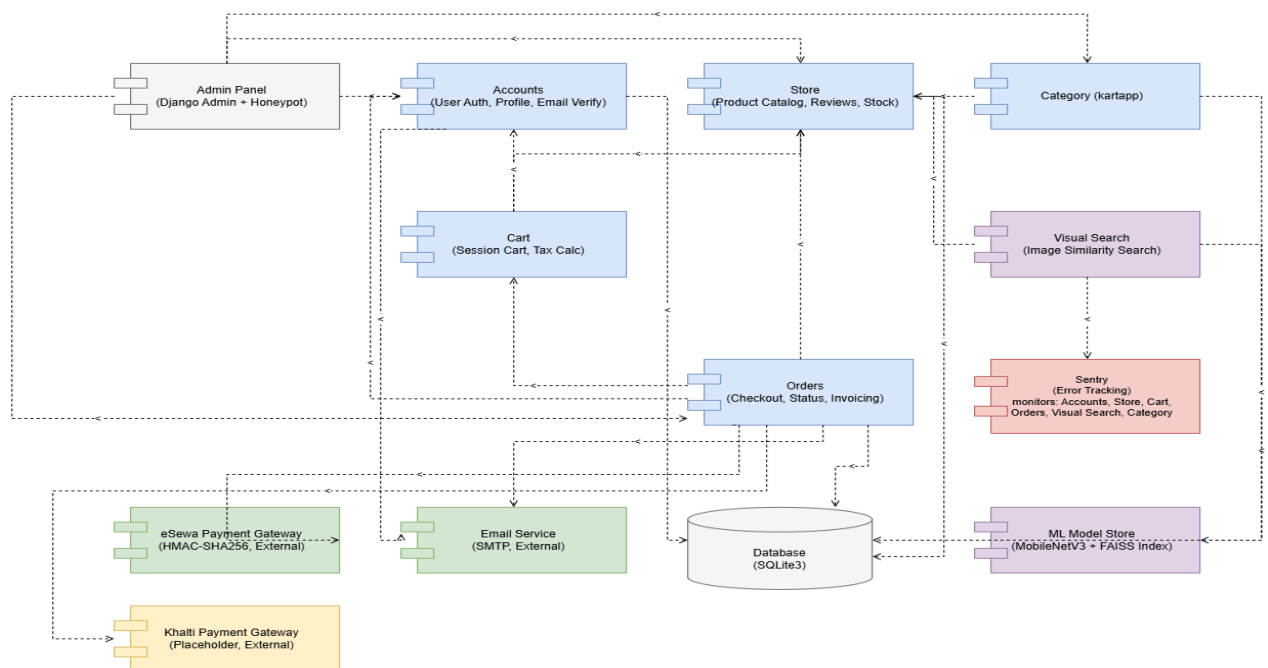


Fig 4.5: Component Diagram of Retail Management System

The component diagram illustrates the high-level architecture of the Retail Management System by showing the major software components and their dependencies. The system is organized into independent Django applications, including Accounts, Store, Category, Cart, Orders, and Visual Search, each responsible for a specific set of functionalities. These components interact with supporting services such as the SQLite database, email service, payment gateways, and machine learning model store to provide a complete e-commerce solution. This modular architecture promotes separation of concerns, improves maintainability, and allows individual components to be developed and tested independently.

The diagram also depicts the interaction between internal system components and external services. The Orders component communicates with the eSewa Payment Gateway to process online payments and uses the Email Service to send order confirmations and invoices. The Visual Search component accesses the ML Model Store, which contains the MobileNetV3 model and FAISS index for image similarity search. Additionally, Sentry monitors multiple system components for runtime errors and exceptions, enabling efficient debugging and system maintenance. The overall component design ensures scalability, loose coupling, and reliable communication between the various modules of the retail management system.

4.2 Algorithm Details

The Retail Management System incorporates a visual search algorithm that enables users to search for products using images. The system uses the MobileNetV3-Small Convolutional Neural Network (CNN) as a feature extractor through transfer learning to generate numerical feature embeddings for product images. These embeddings are indexed using Facebook AI Similarity Search (FAISS).

4.2.1 System Architecture

The visual search module of the Retail Management System is designed using a two-stage architecture consisting of an offline indexing pipeline and an online query pipeline.

The offline indexing pipeline is responsible for preparing the product database for visual search. It processes each product image, extracts its visual features using the MobileNetV3-Small convolutional neural network, and converts these features into numerical

embedding vectors. The generated feature vectors are normalized and stored in a FAISS index, which serves as the searchable database for image retrieval.

The online query pipeline is executed whenever a user performs a visual search by uploading an image. The uploaded image undergoes the same preprocessing and feature extraction steps as the product images in the offline pipeline. The resulting feature vector is normalized and compared with the indexed product embeddings stored in the FAISS index. Based on cosine similarity, the system retrieves the most visually similar products. Finally, the corresponding product information is fetched from the database and returned to the user as search results.

Both pipelines use the same preprocessing operations and the same MobileNetV3-Small feature extraction model. This ensures that the embeddings generated during indexing and querying exist in the same feature space, allowing accurate and consistent similarity comparison.

The overall architecture minimizes the computational workload during user searches because feature extraction for the product catalog is performed only once during indexing, while real-time queries require only feature extraction for the uploaded image and a similarity search within the existing index.

The main components of the visual search architecture are:

- **Image Preprocessing**
 - Resizes and normalizes images to match the input requirements of the CNN model.
- **Feature Extraction**
 - Uses the MobileNetV3-Small model to generate a compact feature embedding representing each image.
- **Feature Normalization**
 - Applies L2 normalization to produce unit-length feature vectors suitable for cosine similarity.
- **FAISS Index**
 - Stores normalized feature vectors and performs efficient similarity searches.

- **Product Database**
 - Stores product details such as name, price, description, and image information.
- **Visual Search Module**
 - Matches the uploaded image against the indexed feature vectors and retrieves the most similar products.
- **Result Generation**
 - Retrieves the corresponding product records and presents the matched products to the user.

4.2.2 Algorithm Overview

The system performs content-based image retrieval (CBIR). For every product image, a 576-dimensional embedding vector is computed offline and stored in a FAISS index. At query time, the uploaded image is mapped to the same embedding space and the k product vectors with the highest cosine similarity to the query vector are returned, in rank order.

The end-to-end retrieval function is:

$$R(q) = \text{argtop}K_i (q^T x_i)$$

where q is the query embedding and x_i are the indexed product embeddings.

4.2.2.1 MobileNetV3-Small (CNN feature extractor)

MobileNetV3-Small is a compact convolutional neural network designed for mobile and embedded inference. In this project it is used purely as a transfer-learning feature extractor: the original 1000-way classification head is replaced with nn.Identity, so the output of the forward pass is the 576-dimensional activation of the penultimate layer.

Selection of MobileNetV3-Small

- Small model size and low CPU latency
- Pretrained on ImageNet1K_V1, the learned filters already capture low- and mid-level visual features (edges, textures, colour patterns) that transfer to product imagery.
- The 576-D output is compact, which keeps FAISS search cheap and the on-disk index small.

Image pre-processing.

- The same pipeline is applied to both indexed product images and uploaded query images:

- Resize the shorter side to 256 pixels, preserving aspect ratio.
- Center-crop to 224×224 .
- Convert to a float tensor in $[0, 1]$.
- Normalise per channel using the ImageNet statistics mean = $[0.485, 0.456, 0.406]$ and standard deviation = $[0.229, 0.224, 0.225]$.

Forward pass

$$v = f\theta(I)$$

where,

- $f\theta$ = fine-tuned MobileNetV3-Small model
- I = input image
- v = 576-dimensional feature embedding

L2 normalisation

Before being passed to FAISS, every embedding is divided by its Euclidean norm so that its magnitude becomes 1.

$$\hat{v} = v / \|v\|_2, \text{ with } \|\hat{v}\|_2 = 1$$

After this normalisation, the dot product between two vectors is mathematically equal to their cosine similarity, so FAISS can be queried with a plain inner-product index without any extra distance conversion.

4.2.2.2 FAISS IndexFlatIP (similarity search)

FAISS is a library for efficient similarity search over dense vectors developed by Facebook AI Research. The specific index used here is IndexFlatIP, which performs an exact, brute-force k-NN search over dense vectors using the inner product (dot product) as the similarity metric.

Selection of FAISS IndexFlatIP

- The catalogue is small (the project bundles a curated subset of products), so the cost of an exact search is trivial.
- The user expects the nearest matches, not an approximate recall trade-off, so the simplest and most correct choice is the right one.
- Using inner product on L2-normalised vectors automatically gives cosine similarity, which is the standard metric for visual embeddings.

Similarity Computation

$$s(q, x_i) = q^T x_i = \sum_{j=1}^{576} q_j x_{ij}$$

Where,

$s(q, x_i)$ = similarity score between the query image and the i th product image

q = normalized query image embedding

x_i = normalized embedding of the i th indexed product

q_j = j th element of the query embedding

x_{ij} = j th element of the i th product embedding

The result ranges in $[-1, 1]$ after L2 normalisation.

Top-K retrieval rule

$$R(q) = \text{argtop}_K s(q, x_i)$$

Where,

$R(q)$ = set of the top K retrieved products

q = normalized query image embedding

x_i = normalized embedding of the i th indexed product

$s(q, x_i)$ = similarity score between the query image and the i th product image

K = number of products returned by the retrieval system; 3

Index construction

$$X = [\hat{x}_1; \hat{x}_2; \dots; \hat{x}_N] \in \mathbb{R}^{(N \times 576)}$$

stored in FAISS, paired with a JSON `id_map` that lists product primary keys in the same order.

4.2.3 Data Acquisition

4.2.3.1 Dataset Source

To improve the feature extraction capability of the visual search model, the Fashion Product Images (Small) dataset obtained from Kaggle was used for fine-tuning the pretrained MobileNetV3-Small model. The dataset contains thousands of fashion product images

belonging to different categories, making it suitable for transfer learning in fashion-related image retrieval tasks.

The pretrained model, originally trained on the ImageNet dataset, was further fine-tuned on this fashion dataset so that it could learn domain-specific visual features such as clothing styles, colors, textures, and product patterns. After fine-tuning, the model was integrated into the Retail Management System as the feature extraction component of the visual search module.

Dataset Source

- Dataset Name: Fashion Product Images (Small)
- Platform: Kaggle
- URL: <https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images-small>
- Purpose: Fine-tuning the MobileNetV3-Small model for fashion image retrieval.

4.2.3.2 Dataset Description

The Fashion Product Images (Small) dataset contains product images along with descriptive metadata for each fashion item. The metadata provides information about product categories and characteristics, enabling the model to learn discriminative visual features during training.

Attribute	Description
id	Unique product identifier
gender	Target gender of the product
masterCategory	Main product category

subCategory	Product subcategory
articleType	Specific type of product
baseColour	Primary product color
season	Suitable season
year	Manufacturing or release year
usage	Intended usage of the product
productDisplayName	Product name

Table 4.1: Dataset Attribute Description

The corresponding image for each record is stored separately and is associated with the metadata using the product ID.

CHAPTER FIVE

IMPLEMENTATION AND TESTING

5.1 Implementation

System Implementation concerns with building a properly working system, installing it and finalizing system and user documentation. Implementation also involves closing down projects. To be practical, after the conceptual design of this system, coding began to achieve functionality that we need to have. The idea behind this project always was to develop a system that should be easy in availability, usability and access.

5.1.1 Tools Used

The development of the Retail Management System required a selection of reliable, scalable, and widely adopted development tools. Each tool was chosen to support specific functional requirements of the system, from user interface development to backend processing, database management, and version control. The major tools used in the project are described below.

Front End

The user interface is constructed using HTML5 and CSS3 for semantic structure and responsive styling, enhanced by the Bootstrap 5 framework to ensure mobile compatibility and consistent design patterns. Dynamic content rendering is achieved through the Django Template Language (DTL), which seamlessly injects server-side data into static pages, while lightweight JavaScript handles client-side interactions such as form validation, dynamic cart updates, and asynchronous image uploads without requiring a separate frontend framework.

Back End

The core application logic is built on Python utilizing the Django 5.2.5 web framework, which follows the Model-View-Template (MVT) architecture to manage routing, authentication, and business rules efficiently. The backend relies on Django's robust ORM for database interactions, session management for cart persistence.

Database

SQLite3 serves as the primary relational database management system, chosen for its lightweight, serverless architecture that simplifies development and testing environments. Data integrity and relationships are managed through the Django ORM, which abstracts raw SQL queries into Python objects, allowing for efficient schema migrations, atomic transactions during order processing, and complex filtering operations for product catalogs without direct database administration overhead.

Development and Collaboration Tools

The team utilizes Git and GitHub for distributed version control, enabling branch management, code reviews, and merge conflict resolution throughout the development lifecycle. Draw.io is employed for creating technical documentation, including UML diagrams and system architecture flows, while Slack facilitates real-time team communication and sprint coordination. Mailtrap is integrated into the development environment to capture and inspect outgoing emails, ensuring that transactional notifications like order confirmations and password resets function correctly without sending actual emails to users.

Additional Tools and Technologies

Sentry is integrated for real-time error tracking and performance monitoring, automatically capturing exceptions and stack traces to aid in rapid debugging and system stability maintenance. Postman is used for manually testing API endpoints and verifying the request-response cycles of external integrations like the eSewa payment gateway.

5.1.2 Implementation Details of Modules

This project can be decomposed into the following modules:

1. Account Module

Implements a custom `AbstractBaseUser` model for email-based authentication, featuring secure password hashing (PBKDF2), email verification tokens, and session management with

auto-logout logic for inactive users. It handles role-based access control distinguishing between customers and superadmins. The module sample code is provided in **Appendix A.1**, while the design is presented in **Appendix B.1**.

2. Store Module

Manages the product catalog using normalized database models for Categories, Products, and Variations (size/color). It implements dynamic filtering logic using Django Q objects for multi-criteria search (price range, category, attributes) and handles image uploads/resizing via Pillow for consistent gallery display. The module sample code is provided in **Appendix A.2**, while the design is presented in **Appendix B.2**.

3. Cart Module

Utilizes Django sessions to persist shopping cart data without immediate database writes, reducing load. It calculates real-time subtotals and taxes (2%) on the fly, manages quantity updates, and validates stock availability before allowing checkout progression. The module sample code is provided in **Appendix A.3**, while the design is presented in **Appendix B.3**.

4. Orders Module

Orchestrates the checkout workflow using atomic database transactions (`transaction.atomic`) to ensure order creation, stock reduction, and payment logging occur simultaneously. It integrates the eSewa payment gateway by generating HMAC-SHA256 signatures for request security and verifying callback signatures to prevent fraud. The module sample code is provided in **Appendix A.4**, while the design is presented in **Appendix B.4**.

5. Visual Search Module

Operates as a standalone utility that loads a pre-trained MobileNetV3 model to extract 576-dimensional feature vectors from product images. It builds and queries a FAISS index file (`index.faiss`) using cosine similarity to retrieve top-matching product IDs, which are then fetched from the main database for display. The module sample code is provided in **Appendix A.5**, while the design is presented in **Appendix B.5**.

6. Reviews Module

Enforces a "verified purchase" constraint by checking the OrderProduct table before allowing review submission. It calculates weighted average ratings dynamically and stores user feedback linked specifically to completed orders to maintain data integrity.

7. Admin Module

Leverages Django's built-in administration interface, customized with ModelAdmin classes to provide a secure dashboard for superusers. It implements strict access controls to manage the entire product lifecycle (CRUD operations for products, categories, and variations) and oversees order fulfillment by allowing status transitions (New /Accepted /Completed). The module also includes tools for moderating user reviews, managing inventory levels, and triggering the build_index management command to synchronize the AI visual search engine with the latest catalog updates

5.2 Testing

Testing is an important phase in system development that ensures the system functions correctly, efficiently, and securely according to the specified requirements, and it helps in identifying errors, improving performance, and ensuring reliability.

5.2.1 Test Cases for Unit Testing

Unit testing validates the performance of individual components before full system integration .

Table 5.1: Test for Account Module

Test Case ID	Test Description	Input Data	Expected Result	Actual Result	Remarks
TC-AC-01	Register new user	Valid name, email, password	User account created,	User account created, email sent	Pass

			verification email sent		
TC-AC-02	Login with valid credentials	Registered email, correct password	User logged in, redirected to dashboard	User logged in successfully	Pass
TC-AC-03	Login with invalid credentials	Valid email, wrong password	Error: "Invalid email or password" displayed	Error message displayed	Pass
TC-AC-04	Password reset request	Registered email address	Reset link generated and emailed	Reset link sent successfully	Pass
TC-AC-05	Email verification	Valid unique token from email	Account status changed to 'Active'	Account verified successfully	Pass

Table 5.2: Test for Store & Product Module

Test Case ID	Test Description	Input Data	Expected Result	Actual Result	Remarks
TC-ST-01	Add new product	Admin enters details, uploads image	Product saved, visible in catalog	Product added successfully	Pass

TC-ST-02	Filter by category	Select "Men's Clothing"	Only products in that category shown	Correct filtered list displayed	Pass
TC-ST-03	Search by keyword	Enter "Blue T-Shirt"	Products matching text returned	Relevant products found	Pass
TC-ST-04	View product variations	Select Size: L, Color: Red	Price and stock update for specific SKU	Variation details updated	Pass
TC-ST-05	Handle out-of-stock item	Select quantity > available stock	"Out of Stock" message displayed	Error message shown correctly	Pass

Table 5.3: Test for Cart Module

Test Case ID	Test Description	Input Data	Expected Result	Actual Result	Remarks
TC-CR-01	Add item to cart	Product ID, Quantity: 1	Item added to session cart	Item added successfully	Pass
TC-CR-02	Update quantity	Change quantity from 1 to 3	Subtotal recalculated (Price × 3)	Subtotal updated correctly	Pass

TC-CR-03	Remove item from cart	Click "Remove" button	Item deleted from session	Item removed successfully	Pass
TC-CR-04	Calculate tax	Cart subtotal \$100	Tax (2%) added, Total = \$102	Total calculated as \$102	Pass
TC-CR-05	Persist cart on refresh	Add items, refresh page	Cart items remain visible	Cart data persisted	Pass

Table 5.4: Test for Order & Payment Module

Test Case ID	Test Description	Input Data	Expected Result	Actual Result	Remarks
TC-OR-01	Create order record	Valid cart, shipping info	Order ID generated, status "New"	Order created successfully	Pass
TC-OR-02	Reduce stock on order	Confirm order placement	Product stock decreased by order qty	Stock reduced correctly	Pass
TC-OR-03	Generate eSewa signature	Transaction ID, Amount	Valid HMAC-SHA 256 signature generated	Signature generated	Pass

TC-OR-04	Verify payment callback	Valid response from eSewa	Order status updated to "Confirmed"	Payment verified, status updated	Pass
TC-OR-05	Handle payment failure	Invalid signature/Failure code	Order status "Failed", stock restored	Failure handled, stock restored	Pass

Table 5.5: Test for Visual Search Module

Test Case ID	Test Description	Input Data	Expected Result	Actual Result	Remarks
TC-VS-01	Upload valid image	JPEG file < 5MB	Image accepted, processing starts	Upload successful	Pass
TC-VS-02	Reject invalid format	.TXT or .EXE file	Error: "Invalid file type" displayed	Error message shown	Pass
TC-VS-03	Extract feature vector	Valid product image	576-dim vector generated	Vector generated successfully	Pass
TC-VS-04	Retrieve similar products	Query vector vs FAISS index	Top-3 similar product IDs returned	3 matches found	Pass

TC-VS-05	Handle no matches	Image with no catalog similarity	Message: "No similar items found"	Fallback message displayed	Pass
----------	-------------------	----------------------------------	-----------------------------------	----------------------------	------

5.2.2 System Testing

System testing validates the complete workflow of the Retail Management System by ensuring proper interaction between the frontend, backend, database, and implemented algorithms.

Table 5.6: System Test Cases

Test Case ID	Test Scenario	Steps Executed	Expected Result	Actual Result	Status
TC-SYS-01	End-to-End Purchase Flow	1. User registers & logs in. 2. Searches via image upload. 3. Adds similar products to cart. 4. Proceeds to checkout with eSewa. 5. Completes payment.	Order is created, stock is reduced, payment status is "Success", and confirmation email is received.	Order created, stock updated, payment verified, email sent.	Pass
TC-SYS-02	Inventory Synchronization	1. Admin sets product stock to 5. 2. Two users simultaneously attempt to buy 3 units each.	First transaction succeeds (stock becomes 2); second transaction fails with "Out of Stock" error.	Race condition handled; stock never went negative.	Pass

TC-SYS-03	Visual Search Integration	1. User uploads a query image. 2. System processes image via MobileNetV3. 3. Results are displayed from FAISS index.	Top-3 visually similar products appear within 2 seconds with correct details fetched from DB.	Results displayed accurately within the time limit.	Pass
TC-SYS-04	Order Status Lifecycle	1. Admin views "New" order. 2. Admin changes status to "Accepted". 3. Admin marks as "Completed".	Status updates at each step; customer receives email notification for each transition.	Status flow working; emails triggered correctly.	Pass
TC-SYS-05	Security & Access Control	1. Guest user tries to access /admin/ . 2. A regular user tries to access another user's order details.	Guest redirected to login; Regular user receives "Access Denied" (403).	Unauthorized access successfully blocked.	Pass
TC-SYS-06	Payment Gateway Failure	1. Initiate checkout. 2. Simulate eSewa callback with invalid HMAC signature.	System rejects transaction, logs security alert, and restores cart items.	Transaction rejected; security log updated.	Pass

5.3 Result Analysis

The Retail Management system was tested through unit testing and system testing to ensure that all functional modules work as intended. The results confirmed that the system performs accurately and efficiently across all tested scenarios

5.3.1 Evaluation Methodology

The performance of the visual search system was evaluated using a labelled query dataset. Each query image was processed using the proposed visual search pipeline, which consists of feature extraction using the fine-tuned MobileNetV3-Small model followed by similarity search using FAISS. The retrieved products were compared with the ground-truth product labels to evaluate the effectiveness of the retrieval system.

The evaluation was performed using the following metrics:

- Top-1 Accuracy
- Top-3 Accuracy
- Precision@3
- Average Retrieval Time

5.3.2 Evaluation Metrics

Top-1 Accuracy

Top-1 Accuracy measures the percentage of query images for which the correct product is retrieved as the first result.

$$\text{Top-1 Accuracy} = \frac{H_1}{N} \times 100\%$$

where

- H_1 = Number of queries where the first retrieved product is relevant
- N = Total number of query images

Top-3 Accuracy

Top-3 Accuracy measures the percentage of queries for which the correct product appears within the first three retrieved results.

$$\text{Top-3 Accuracy} = \frac{H_3}{N} \times 100\%$$

where

- H_3 = Number of queries where at least one relevant product appears in the top three results
- N = Total number of query images

Precision@3

Precision@3 evaluates the proportion of relevant products among the top three retrieved results.

$$\text{Precision@3} = \frac{1}{N} \sum_{q=1}^N \frac{|A(q) \cap R(q)|}{3} \times 100\%$$

where

- $A(q)$ = Top three retrieved products
- $R(q)$ = Relevant products for query q
- N = Total number of queries

Average Retrieval Time

Average Retrieval Time measures the average time required to retrieve similar products for a query image.

$$\text{Average Retrieval Time} = \frac{\sum_{q=1}^N t_q}{N}$$

where

- t_q = Retrieval time for query q
- N = Total number of queries

5.3.3 Experimental Results

The visual search system was evaluated using **145 query images** from the evaluation dataset. The performance was measured using Top-1 Accuracy, Top-3 Accuracy, Precision@3, and Average Retrieval Time.

Metric	Result
Top-1 Accuracy	96.55 %
Top-3 Accuracy	98.62 %
Precision@3	32.87%
Average Retrieval Time	9.33 ms

Table 5.7: Performance Evaluation Results of the Visual Search module

5.3.4 Discussion

The evaluation results indicate that the proposed visual search system is capable of retrieving visually similar products efficiently. The Top-1 Accuracy shows how often the correct product is retrieved as the highest-ranked result, while the Top-3 Accuracy measures the ability of the system to retrieve the correct product within the first three results.

Precision@3 evaluates the relevance of the retrieved products and reflects the quality of the search results presented to the user. In addition, the Average Retrieval Time demonstrates the computational efficiency of the system, indicating that visual search can be performed within a short response time suitable for real-time applications.

Overall, the combination of the fine-tuned MobileNetV3-Small model and FAISS similarity search provides an effective and efficient visual search solution for the Retail Management System.

CHAPTER SIX

CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

The development of the Retail Management System successfully demonstrates the viability of integrating deep learning capabilities into a traditional e-commerce framework. By leveraging Django's robust MVT architecture, the project delivers a secure, scalable, and modular platform that handles essential retail operations including user authentication, inventory management, and order processing with data integrity. The implementation of custom user models and atomic database transactions ensures that critical business logic, such as stock reduction and payment verification, operates reliably even under concurrent user loads, providing a stable foundation for online retail operations. A significant achievement of this project is the seamless incorporation of an AI-powered visual search engine using MobileNetV3 and FAISS. This feature transforms the user experience by allowing customers to discover products through images rather than relying solely on text keywords, effectively bridging the gap between visual inspiration and purchase intent. The system's ability to extract 576-dimensional feature vectors and perform millisecond-level similarity searches proves that advanced computer vision techniques can be practically deployed in web applications without requiring excessive computational resources or custom model training from scratch. The security and financial integrity of the platform are upheld through rigorous implementation of industry-standard protocols, specifically the HMAC-SHA256 signature verification for the eSewa payment gateway. This ensures that all transactions are authenticated and tamper-proof, addressing a critical concern in digital commerce. Furthermore, the inclusion of automated email notifications, verified purchase reviews, and role-based access control creates a comprehensive ecosystem that fosters trust between customers and administrators, essential for the success of any online marketplace. Throughout the development lifecycle, the adoption of an Agile methodology allowed for iterative refinement of features, ensuring that each module from the shopping cart to the admin dashboard met functional requirements before integration. The extensive testing phases, including unit tests for individual components and system tests for end-to-end workflows,

validated the system's reliability and responsiveness. The result is a cohesive platform that not only meets current retail needs but also establishes a flexible architecture capable of adapting to future technological advancements and market demands. In conclusion, the Retail Management system serves as a proof-of-concept for next-generation retail solutions in emerging markets, combining localized payment integrations with global AI standards. It effectively addresses the limitations of traditional text-based search and fragmented management tools by offering a unified, intelligent, and secure platform. While the current implementation focuses on web-based interactions, the modular design and clear separation of concerns provide a solid groundwork for future expansions, positioning the system as a viable solution for small to medium-sized enterprises seeking to modernize their retail operations.

6.2 Future Recommendations

1. Mobile Application Development: Develop native iOS and Android applications using frameworks like Flutter or React Native to consume the existing Django REST API, enabling push notifications, offline cart access, and a superior mobile-first shopping experience for users on the go.

2. Advanced Retail Owner Analytics: Enhance the admin dashboard with a dedicated "Retail Owner Portal" featuring real-time sales visualization, inventory forecasting using time-series analysis, and customer behavior heatmaps to support data-driven decision-making.

3. Model Fine-Tuning for Domain Specificity: Retrain or fine-tune the MobileNetV3 model on a specialized fashion and retail dataset (e.g., DeepFashion) to improve feature extraction accuracy for specific textures, patterns, and styles, thereby increasing visual search precision.

4. Multi-Warehouse & Supply Chain Integration: Expand the inventory module to support multiple warehouse locations, automated low-stock alerts, and integration with third-party logistics (3PL) providers for streamlined shipping and real-time tracking.

5. AI-Driven Personalization Engine: Implement a collaborative filtering or hybrid recommendation system that analyzes user browsing history and purchase patterns to suggest personalized products on the homepage and via targeted email marketing campaigns.

References

- [1] IMNepal, “The Growth of E-commerce in Nepal: Opportunities and Challenges,” *IMNepal*, 2023. [Online]. Available: <https://www.imnepal.com/growth-e-commerce-in-nepalopportunities-challenges/>.
- [2] International Trade Administration, “Nepal – eCommerce,” *ITA*, 2024. [Online]. Available: <https://www.trade.gov/country-commercial-guides/nepal-ecommerce>. [Accessed: Feb. 6, 2026].
- [3] A. Maulak, “The Rise of E-commerce in Nepal,” *Ayush Maulak Blog*, 2023. [Online]. Available: <https://ayushmaulak.com.np/rise-of-e-commerce-in-nepal/>.
- [4] R. Robu, “Understanding and Implementing MobileNetV3,” *Medium*, 2023. [Online]. Available: <https://medium.com/@RobuRishabh/understanding-and-implementing-mobilenetv3-422bd0bdfb5a>. [Accessed: Jul. 3, 2026]
- [5] D. Bytes, “Similarity Search with FAISS: A Practical Guide to Efficient Indexing and Retrieval,” *Medium*, 2024. [Online]. Available: <https://medium.com/@devbytes/similarity-search-with-faiss-a-practical-guide-to-efficient-indexing-and-retrieval-e99dd0e55e8c>. [Accessed: Jun. 30, 2026].
- [6] Cloco Nepal, “E-commerce in Nepal: Trends and Challenges,” *Cloco Blog*, 2024. [Online]. Available: <https://cloco.com.np/blog/ecommerce-nepal-2025-trends>.
- [7] Agile SDLC (Software Development Life Cycle), GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/software-engineering/agile-sdlc-software-development-life-cycle/>. [Accessed: 17-Feb-2026].
- [8] P. Aggarwal, *Fashion Product Images (Small)*, Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images-small>. [Accessed: Jul. 6, 2026].
- [9] PyTorch, “MobileNetV3,” *Torchvision Documentation*. [Online]. Available: <https://docs.pytorch.org/vision/stable/models/mobilenetv3.html>. [Accessed: Jul. 12, 2026].
- [10] J. Johnson, M. Douze, and H. Jégou, “FAISS: A Library for Efficient Similarity Search,” *Meta Engineering*, Mar. 29, 2017. [Online]. Available: <https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/>. [Accessed: Jul. 30, 2026].

Appendices

```
1 class MyAccountManager(BaseUserManager):
2
3     # Create a normal User
4     def create_user(self, first_name, last_name, username, email, password=None):
5
6         if not email:
7             raise ValueError("User must have an valid email address")
8         if not username:
9             raise ValueError("User must have an username")
10
11         user = self.model(
12             email = self.normalize_email(email),
13             username = username,
14             first_name = first_name,
15             last_name = last_name,
16         )
17         user.set_password(password)
18         user.save(using=self._db)
19         return user
20
21     # Create a super user
22     def create_superuser(self, first_name, last_name, username, email, password):
23
24         user = self.create_user(
25             email=self.normalize_email(email),
26             username = username,
27             password= password,
28             first_name = first_name,
29             last_name = last_name,
30         )
31         user.is_active = True
32         user.is_staff = True
33         user.is_superuser = True
34
35         user.save(using=self._db)
36         return user
37
38 class Account(AbstractBaseUser, PermissionsMixin):
39     first_name = models.CharField(max_length=50)
40     last_name = models.CharField(max_length=50)
41     username = models.CharField(max_length=50, unique=True)
42     email = models.EmailField(max_length=254, unique=True)
43     phone = models.CharField(max_length=50)
44
45     # required
46     is_active = models.BooleanField(default=False)
47     is_staff = models.BooleanField(default=False)
48     is_superuser = models.BooleanField(default=False)
49
50     date_joined = models.DateTimeField(auto_now_add=True)
51
52     USERNAME_FIELD = "email"
53     REQUIRED_FIELDS = ['username', 'first_name', 'last_name']
54
55     objects = MyAccountManager()
56
57     def __str__(self):
58         return self.email
```

Appendix A1 : Custom User Model Sample Code

Sign in

[Forgot password?](#)

Don't have account? [Sign up](#)

Sign up

UserName *

Phone Number *

Email Address *

We'll never share your email with anyone else.

Create Password *

Confirm Password *

Have an account? [Log In](#)

Appendix B1 : User Authentication Sample Design

```

1  from django.urls import path
2  from store import views
3
4  urlpatterns = [
5      path('',views.StoreView,name='store'),
6      path('category/<slug:category_slug>/',views.StoreView,name='product_by_category'), #dynamic_Url
7      path('category/<slug:category_slug>/<slug:product_slug>/',views.product_detail,name='product_detail'),
8      path('search/',views.search,name='search'),
9      path('submit_review/<int:product_id>/', views.submit_review, name='submit_review'),
10 ]

```

Appendix A2 : Store Sample Code

ClickShop

All category

Store

Search

Welcome guest!
Sign in | Register

SMART SHOPPING

Find outfits faster with category picks and visual search.

Browse curated collections, compare styles, and discover your next favorite look in seconds.

Browse categories

View cart

Shop by Category



JACKETS



T-SHIRTS



JEANS



SHIRTS



SHOES



BAG

SHOP COLLECTION

Minimal storefront, modern browsing, and products that stay easy to compare.

Explore curated items with straightforward filters, clear product cards, and a simpler shopping flow.

ITEMS FOUND

25

CATEGORIES

6

SIZES

7

Filters

Keep browsing focused and simple.

Categories

All Items

Jackets

T-Shirts

Jeans

Shirts

Shoes

Bag

Sizes

41

42

Default

Extra Large

Large

Medium

Small

Store collection

25 Items found

Explore products in a clean grid with simple details and faster comparison.

JEANS

IN STOCK

Atx-Jean

★★★★☆ 1 reviews

Rs 20

View details

SHIRTS

IN STOCK

Blue Shirt

★★★★☆ 0 reviews

Rs 200

View details

T-SHIRTS

IN STOCK

Graphic T-shirt

★★★★☆ 0 reviews

Rs 299

View details

Atx-Jean

★★★★☆ 1 reviews

Rs 20

Classic denim built for comfort and everyday style and Jeans that shows class

Choose Color

Select

Select Size

Select

Add to Cart

Write Your Review

How do you rate this product?

★★★★★

Review Title

Enter your thoughts

Appendix B2 : Store Sample Design

50

```

1 def cart(request, total = 0, quantity = 0, tax = 0, grand_total = 0, cart_items = None):
2
3     if request.user.is_authenticated:
4         cart_items = CartItem.objects.filter(user = request.user, is_active = True)
5
6         for item in cart_items:
7             total += (item.product.price * item.quantity)    # every product in cart's
8             quantity += item.quantity
9         tax = (2 * total)/100
10        grand_total = total + tax
11
12        context = {
13            'total':total,
14            'quantity':quantity,
15            'cart_items':cart_items,
16            'tax': tax,
17            'grand_total' : grand_total
18        }
19        return render (request, 'carts/cart.html', context)
20
21    else:
22        messages.info(request, "Please Sign in before adding to cart")
23        return redirect('signin')

```

Appendix A3 : Cart Sample Code

PRODUCT	QUANTITY	PRICE	
 White Oxford Shirt Color : White Size : Medium	- 1 +	Rs 699 Rs 699 each	Remove

Sub Total: Rs 699

Tax: Rs 13.98

Total: Rs 712.98




Checkout

Continue Shopping

Appendix B3 : Cart Sample Code

```

1 from django.urls import path
2 from orders import views
3
4 urlpatterns = [
5
6     path('place_order/', views.Place_Order, name='place_order'),
7
8     path('payment/esewa/<str:order_number>', views.eSewa_payment, name='esewa_payment'),
9     path('payment/khalti/<str:order_number>', views.Khalti_payment, name='khalti_payment'),
10    path('payment/cod/<str:order_number>', views.COD_payment, name='cod_payment'),
11
12    path('payment_success/', views.Payment_Success, name='payment_success'),    # required only for verification
13    path('payment_failure/', views.Payment_Failure, name='payment_failure'),
14
15    path('invoice/<str:order_number>', views.order_invoice, name='order_invoice'),
16
17 ]

```

Appendix A4 : Order & Payment Sample Code

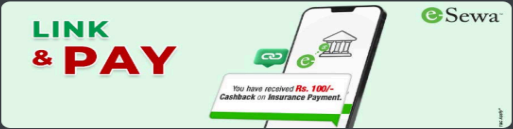
Shipping Address SUSHANK LAMSAL Gongabu Kathmandu , Bagmati , Nepal sushanklamsal@gmail.com 9812345678	Sub Total : Rs 699.0 Tax: Rs 13.98 Total: Rs 712.98  Make Payment						
Payment Method 							
Review products <table border="1"> <thead> <tr> <th>PRODUCT</th> <th>QUANTITY</th> <th>PRICE</th> </tr> </thead> <tbody> <tr> <td>  White Oxford Shirt Color : White Size : Medium </td> <td>1</td> <td> Rs 699 Rs 699 each </td> </tr> </tbody> </table>	PRODUCT	QUANTITY	PRICE	 White Oxford Shirt Color : White Size : Medium	1	Rs 699 Rs 699 each	
PRODUCT	QUANTITY	PRICE					
 White Oxford Shirt Color : White Size : Medium	1	Rs 699 Rs 699 each					


ePAYTEST

Total Amount

NPR. 712.98

Product Amount	699.00
Tax Amount	13.98
Total Amount	712.98



Sign in to your account









I'm not a robot

This site is exceeding reCAPTCHA Enterprise free quota

LOGIN

Forgot Password?

Don't have an account? [Register](#)

CANCEL PAYMENT



Payment Successful

[Shop more](#)

Order Invoice

Invoiced To:

SUSHANK LAMSAL
Gongabu
Kathmandu, Bagmati
Nepal
Phone: 9812345678
Email: sushanklamsal@gmail.com

Order Details:

Order Number:
20260731158

Payment Method:
eSewa

Status:
COMPLETE

Product	Quantity	Price
White Oxford Shirt Color: White Size: Medium	1	Rs. 699 each

Sub Total: Rs. 699

Tax: Rs. 13.98

Total: Rs. 712.98

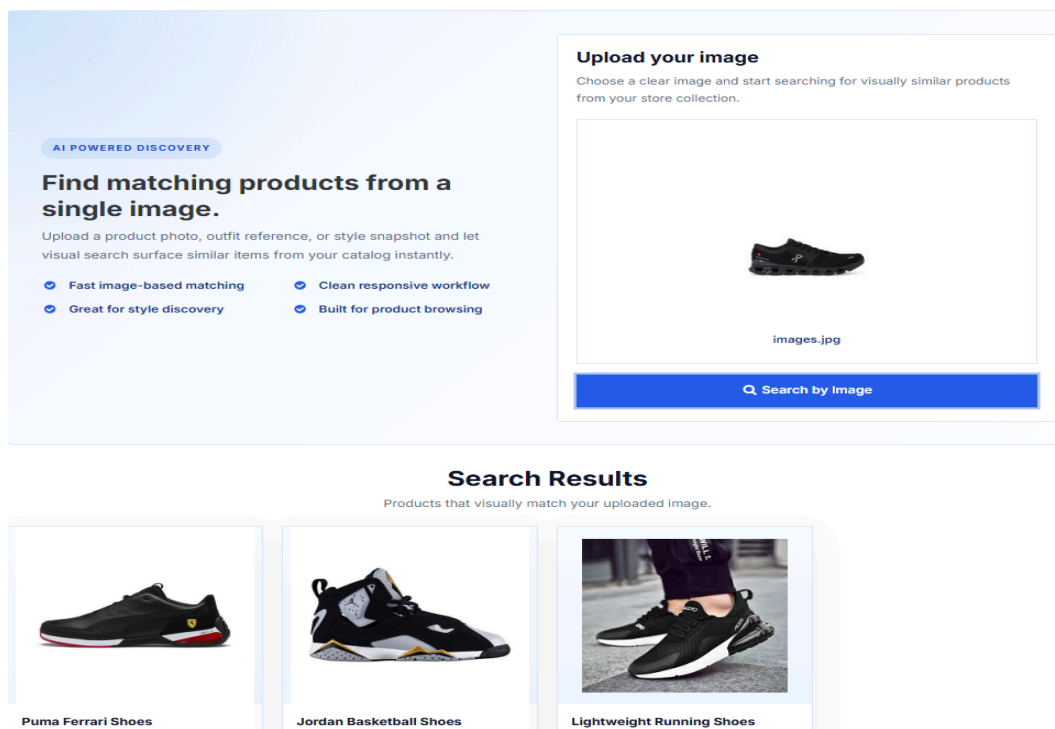
Appendix B4 : Order & Payment Sample Design

```

1  def _build_model() -> nn.Module:
2
3      model = models.mobilenet_v3_small(weights=models.MobileNet_V3_Small_Weights.IMAGENET1K_V1)
4      model.classifier = nn.Identity()
5      model.eval()
6      return model
7
8  _model: nn.Module = _build_model()
9
10
11 def extract_features(pil_image: Image.Image) -> np.ndarray:
12
13     img = pil_image.convert("RGB")
14     tensor = _TRANSFORM(img).unsqueeze(0) # shape: (1, 3, 224, 224)
15
16     with torch.no_grad():
17         features = _model(tensor) # shape: (1, 576)
18
19     vec = features.squeeze().numpy().astype(np.float32) # (576,)
20
21     norm = np.linalg.norm(vec)
22     if norm > 0:
23         vec = vec / norm
24     return vec

```

Appendix A5 : Visual Search Sample Code



Appendix B5 : Visual Search Sample Design